Practical 3

Basic Neural Network for Image Classification using a Dummy Image Dataset

Aim

To build, train and evaluate a basic feed-forward (fully connected / Dense) artificial neural network using TensorFlow/Keras that classifies synthetically generated 28×28 grayscale images into three classes — Square, Diagonal, and Horizontal.

Conceptual / Theoretical Background

An Artificial Neural Network (ANN) is a machine-learning model loosely inspired by biological neurons. It is composed of layers of interconnected nodes (neurons); each connection has a learnable weight, and each neuron applies an activation function to the weighted sum of its inputs. A Dense (fully connected) layer connects every neuron in one layer to every neuron in the next.

For image data, a Dense network first needs the 2D image 'flattened' into a 1D vector (a Flatten layer converts a 28×28 image into a 784-length vector), since Dense layers only accept 1D input per sample.

The ReLU (Rectified Linear Unit) activation function, used in the hidden layers, outputs the input directly if it is positive and zero otherwise; it introduces non-linearity so the network can learn complex patterns. The output layer uses Softmax activation, which converts the network's raw scores into a probability distribution over the classes so the predicted class is the one with the highest probability.

Training minimises a loss function — here sparse categorical cross-entropy, appropriate when labels are integers (0, 1, 2) rather than one-hot vectors — using an optimizer (Adam) that iteratively adjusts the network's weights via backpropagation and gradient descent. Training runs for a fixed number of epochs (full passes over the training data), processing the data in batches.

The dataset is split into training, validation and test sets. The training set is used to fit the weights, the validation split (taken from the training data during fit()) monitors performance on unseen data during training to detect overfitting, and the held-out test set gives a final, unbiased measure of how well the model generalises.

Real-World Applications

- Handwritten digit/character recognition (e.g. the classic MNIST digit classifier) is the direct real-world analogue of this synthetic-shape classifier.
- Basic pattern/defect recognition in manufacturing, where simple visual patterns need to be sorted into a small number of categories.
- Simple biometric or gesture recognition systems that classify a small, fixed set of visual patterns.
- As an educational stepping stone before Convolutional Neural Networks (CNNs), which are the standard choice for real-world image classification (covered in Practical 4).

Important Terminology

Term	Meaning
Neuron / Node	The basic computing unit of a neural network; computes a weighted sum of inputs and applies an activation function.

Term	Meaning
Dense (Fully Connected) layer	A layer where every neuron is connected to every neuron in the previous layer.
Flatten layer	Converts a multi-dimensional input (e.g. a 28×28 image) into a 1D vector.
ReLU	Activation function that outputs max(0, x); adds non-linearity to hidden layers.
Softmax	Activation function used in the output layer that converts scores into class probabilities summing to 1.
Loss function	A measure of how far the model's predictions are from the true labels; sparse categorical cross-entropy is used here.
Optimizer (Adam)	The algorithm that updates network weights to minimise the loss during training.
Epoch	One complete pass of the training algorithm over the entire training dataset.
Batch size	The number of samples processed before the model's weights are updated once.
Normalization	Scaling pixel values (here, clipped to the 0–1 range) so the network trains more effectively.
Train/validation/test split	Dividing data into a set to fit the model, a set to monitor training, and a held-out set for final evaluation.

Step-by-Step Working

1. Import NumPy, Matplotlib, scikit-learn's train_test_split, and TensorFlow/Keras layers; set random seeds for reproducibility.
2. Programmatically create a dummy dataset of 28×28 grayscale images: 200 samples each of a bright centre square (Class 0), a bright diagonal line (Class 1), and a bright horizontal line (Class 2), each on a noisy dark background.
3. Visualise sample images from each class to confirm the dataset looks as intended.
4. Normalize pixel values by clipping them to the 0–1 range.
5. Split the dataset into training (80%) and test (20%) sets using stratified sampling so each class is represented proportionally.
6. Build a Sequential Dense network: Flatten → Dense(64, ReLU) → Dense(32, ReLU) → Dense(3, Softmax).
7. Compile the model with the Adam optimizer, sparse categorical cross-entropy loss, and accuracy as the evaluation metric.
8. Train the model for 10 epochs with a batch size of 32, holding out a further 20% of the training data for validation.
9. Plot training vs. validation accuracy across epochs to visualise learning progress.
10. Evaluate the trained model on the held-out test set to obtain the final test loss and accuracy.
11. Predict the class of a few test images and compare the predicted label against the true label.
12. Save the trained model to disk for later reuse.

Python Implementation

1. Imports and reproducibility:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense

np.random.seed(42)
tf.random.set_seed(42)
```

2. Generate the dummy dataset (3 classes):

```
img_size = 28
samples_per_class = 200

images, labels = [], []

# Class 0: bright centre square
for _ in range(samples_per_class):
    img = np.random.rand(img_size, img_size) * 0.2
    img[10:18, 10:18] += 0.8
    images.append(img); labels.append(0)

# Class 1: bright diagonal line
for _ in range(samples_per_class):
    img = np.random.rand(img_size, img_size) * 0.2
    for i in range(img_size):
        img[i, i] += 0.8
    images.append(img); labels.append(1)

# Class 2: bright horizontal line
for _ in range(samples_per_class):
    img = np.random.rand(img_size, img_size) * 0.2
    img[13:15, :] += 0.8
    images.append(img); labels.append(2)

X = np.array(images)
y = np.array(labels)
print("Image data shape:", X.shape)
print("Label shape:", y.shape)
```

3. Normalize and split into train/test sets:

```
X = np.clip(X, 0, 1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Training images:", X_train.shape)
print("Testing images :", X_test.shape)
```

4. Build, compile and train the Dense network:

```
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(3, activation='softmax')
])
model.summary()
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

5. Evaluate and predict:

```
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Loss     : {test_loss:.4f}")
print(f"Test Accuracy : {test_accuracy:.4f}")

class_names = ['Square', 'Diagonal', 'Horizontal']
predictions = model.predict(X_test[:5])

for i in range(5):
    predicted_label = np.argmax(predictions[i])
    true_label = y_test[i]
    print(f"Predicted: {class_names[predicted_label]}  |  True: {class_names[true_label]}")

model.save("basic_dummy_image_model.keras")
print("Model saved successfully.")
```

Explanation of the Code

- The dataset-generation loop builds each 28×28 image as random background noise (0–0.2) with a distinctive bright pattern (+0.8 intensity) added for its class, so the three classes are visually distinguishable but not identical between samples.
- np.clip(X, 0, 1) normalizes all pixel intensities into the 0–1 range expected by the network, which speeds up and stabilises training.
- train_test_split(..., stratify=y) ensures the 80/20 train/test split contains a proportional number of samples from each of the three classes.
- Flatten(input_shape=(28, 28)) turns each 28×28 image into a 784-element vector so it can be fed into the Dense layers.
- The two hidden Dense layers (64 and 32 neurons, ReLU activation) learn increasingly abstract combinations of pixel intensities; the final Dense(3, softmax) layer outputs a probability for each of the three classes.
- model.compile() wires the model to the Adam optimizer, sparse categorical cross-entropy loss (suited to integer class labels), and accuracy tracking.
- model.fit() trains for 10 epochs in batches of 32, reserving 20% of the training data as a validation set to monitor generalisation during training.
- model.evaluate() computes the final loss and accuracy on the completely unseen test set, giving an unbiased performance estimate.
- np.argmax(predictions[i]) picks the class with the highest predicted probability for each test image, which is then compared against the true label.

- model.save() persists the trained weights and architecture to a .keras file so the model can be reloaded without retraining.

Expected Output

A typical run (results vary slightly with random initialisation) produces:

```
Image data shape: (600, 28, 28)
Label shape: (600,)
Training images: (480, 28, 28)
Testing images : (120, 28, 28)

Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 flatten (Flatten)           (None, 784)               0
 dense (Dense)               (None, 64)                50240
 dense_1 (Dense)             (None, 32)                2080
 dense_2 (Dense)             (None, 3)                 99
=================================================================
Total params: 52,419

Epoch 1/10 ... accuracy: 0.55 - val_accuracy: 0.80
...
Epoch 10/10 ... accuracy: 0.99 - val_accuracy: 0.98

Test Loss     : 0.0542
Test Accuracy : 0.9833

Predicted: Square      |  True: Square
Predicted: Diagonal    |  True: Diagonal
Predicted: Horizontal  |  True: Horizontal
Predicted: Square      |  True: Square
Predicted: Diagonal    |  True: Diagonal

Model saved successfully.
```

Conclusion

A basic Dense neural network was successfully trained to classify the three synthetic shape classes with high accuracy, since the patterns are simple and spatially localised. However, because Dense layers treat every pixel independently after flattening, they ignore the 2D spatial structure of the image — a limitation addressed by the Convolutional Neural Network built in Practical 4, which is compared directly against this model.